

پروژه طراحی الگوریتم

هانیه وطنی

1400012268035

○ مراحل انجام پروژه و توضیح مرتبط با هر بخش:

1. کتابخانه‌های مورد نیاز:

دلیل استفاده از هر کتابخانه به صورت کامنت نوشته شده.

```
import networkx as nx # for creating graphs
import matplotlib.pyplot as plt # for visualizing data
import random # for generating random numbers
import os # for saving the data
```

2. کلاس ATM برای موجودیت دستگاه خودپرداز:

برای کلاس ATM باید نام، ویژگی اولویت، ویژگی نود شروع و ویژگی پر یا خالی بودن، همچنین ویژگی زمان refill شدن دوباره دستگاه خودپرداز و لیستی از همسایه‌های ATM را تعریف می‌کنیم. اولویت و نود شروع بودن و همچنین پر بودن دستگاه در حالت اولیه False می‌باشد. زمان پر شدن دستگاه هم نامعلوم (NONE) است. همچنین لیست همسایه‌های ATM در ابتدا خالی است.

```
class ATM:
    def __init__(self, name, priority = False, startATM = False, filled = False):
        self.name = name
        self.priority = priority
        self.startATM = startATM
        self.filled = filled
        self.refilledTime = None
        self.neighbors = [] # list of tuples (neighborName, distance)

    def __str__(self):
        return self.name
```

3. کلاس GRAPH برای موجودیت گراف (map دستگاه‌های خودپرداز):

برای کلاس و موجودیت graph، یک لیست از نودها و یکی هم برای یال‌های در نظر می‌گیریم. که در ابتدا خالی هستند. یک تابع `addNode()` نیز تعریف می‌کنیم. کاربرد این تابع این همانطور که از نامش پیداست، اضافه کردن یک نود به گراف و ساخت لیست خالی از یال‌های این نود است. تابع `addEdge()` نیز با توجه به نامش می‌توان متوجه شد که برای اضافه کردن یک یال مرتبط‌کننده دو نود به گراف است. همچنین چون گراف ما جهت‌دار نیست هر دو نود در لیست همسایه‌های خود اضافه می‌شوند. در آخر نیز تابع `toNetworkx()` برای نمایش گرافیکی گراف استفاده می‌شود.

```
class Graph:
    def __init__(self):
        self.nodes = [] # list of ATM objects
        self.edges = {} # dictionary of edges with weights

    def addNode(self, atm):
        if atm not in self.nodes:
            self.nodes.append(atm)
            self.edges[atm] = []

    def addEdge(self, atm1, atm2, distance):
        if atm1 in self.nodes and atm2 in self.nodes:
            self.edges[atm1].append((atm2, distance))
            self.edges[atm2].append((atm1, distance)) # for undirected graph
            atm1.neighbors.append((atm2.name, distance)) # add (atm2, distance) to atm1's neighbors
            atm2.neighbors.append((atm1.name, distance)) # add (atm1, distance) to atm2's neighbors

    def toNetworkx(self):
        # converts the graph into a Ntetworkx graph for visualization
        G = nx.Graph()

        for atm in self.nodes:
            G.add_node(str(atm), priority = atm.priority, startATM = atm.startATM)

        for atm, connections in self.edges.items():
            for connectedATM, distance in connections:
                G.add_edge(str(atm), str(connectedATM), weight = distance)
        return G
```

4. ساخت تصادفی graph:

در این تابع یک گراف به صورت تصادفی طبق تعداد نودی که تصادفاً ایجاد و به عنوان آرگومان ورودی به تابع داده شده ساخته می‌شود. برای اضافه کردن همسایه‌های رندوم (حداقل 2 و حداکثر 4) ابتدا باید چک کنیم که نود انتخاب شده خود نود فعلی نباشد (نود نمی‌تواند همسایه خود باشد!)، از قبل در لیست همسایه‌های نود فعلی نباشد و همچنین تعداد همسایه‌های نود انتخاب شده کمتر از 4 باشد. سپس از بین نودهای ممکن یک تعداد تصادفی انتخاب و همسایگی ایجاد می‌شود.

```
# function to generate the graph
def generateGraph(numNodes, mindistance = 1, maxdistance = 20, minNeighbors = 2, maxNeighbors = 4):
    graph = Graph()

    # create ATMs
    for i in range(numNodes):
        graph.addNode(ATM(f"{i + 1}"))

    # add random edges ensuring each ATM has between 2 and 4 neighbors
    for atm1 in graph.nodes:
        possibleNeighbors = [
            atm2 for atm2 in graph.nodes
            if atm2 != atm1 and
            atm2 not in [neighbor[0] for neighbor in atm1.neighbors] and
            len(atm2.neighbors) < 4
        ]

        # ensure minNeighbors and maxNeighbors are valid
        if len(atm1.neighbors) < 4 and len(possibleNeighbors) > 0:
            minVal = max(0, minNeighbors - len(atm1.neighbors)) # at least 2 neighbors total
            maxVal = min(len(possibleNeighbors), maxNeighbors - len(atm1.neighbors)) # at most 4 neighbors total

            # randomly select neighbors
            neighbors = random.sample(possibleNeighbors, random.randint(minVal, maxVal))

            for atm2 in neighbors:
                distance = random.randint(mindistance, maxdistance)
                graph.addEdge(atm1, atm2, distance)

    return graph
```

5. نمایش گرافیکی graph:

برای نمایش گرافیکی همان طور که قبلاً گفتیم از graph.toNetworkx() استفاده می‌کنیم. برای نودها بدون اولویت رنگ آبی روشن و برای اولویت‌دارها رنگ آبی را انتخاب می‌کنیم. برای نود شروع نیز یک حلقه‌ی مشکی دور نود می‌کشیم تا نمایش بهتری داشته باشد. بعد از نمایش گرافیکی برنامه باید اتمام یابد پس از exit() در انتهای تابع استفاده می‌کنیم.

```
# function for visualizing the graph
def visualizeGraph(graph):
    nxGraph = graph.toNetworkx()
    positions = nx.spring_layout(nxGraph, k = 5, iterations = 500) # compute positions for nodes

    # draw nodes with priority coloring
    colors = ['blue' if nodeData['priority'] else 'lightblue' for _, nodeData in nxGraph.nodes(data = True)]
    nx.draw(nxGraph, positions, with_labels = True, node_color = colors, node_size = 1500, font_size = 12)

    # draw the start node with a black border
    startNode = next((node for node, data in nxGraph.nodes(data = True) if data['startATM']))
    startColor = 'blue' if nxGraph.nodes[startNode]['priority'] else 'lightblue'
    nx.draw_networkx_nodes(nxGraph, positions, nodelist = [startNode], node_size = 1500, node_color = startColor, edgecolors = 'black', linewidths = 1.3)

    # draw edge labels
    nx.draw_networkx_edge_labels(nxGraph, positions, edge_labels = nx.get_edge_attributes(nxGraph, 'weight'), font_size = 8)

    plt.title("Graph Visualization")
    plt.show()

    exit() # end the program after displaying the graph
```

6. جمع‌آوری اطلاعات کلی graph:

برای کامل‌تر بودن دیدمان به گراف اطلاعات کلی گراف (نام هر ATM و اولویت‌دار بودن یا نبودن و همسایه‌های هر نود) را در LOG فایل ذخیره می‌کنیم.

```
# printing graph's info
def storingGraphsInfo(graph):
    graphInfo = []

    for atm in graph.nodes:
        graphInfo.append((atm.name, atm.priority, atm.neighbors))

    createDataFile(graphInfo)
```

7. پیدا کردن کوتاه‌ترین مسیر از یک نود (ATM) شروع به بقیه نودها (AMTs):

برای پیدا کردن کوتاه‌ترین مسیر از یک نود شروع به بقیه نودها از الگوریتم دایجسترا (Dijkstra) استفاده می‌کنیم. این الگوریتم مرتبه زمانی $O((V + E) \log V)$ است. که در آن V همان تعداد نودهای گراف و E تعداد یال‌های گراف است.

```
# find the shortest path using Dijkstra's algorithm
def findShortestPath(graph, startNode):
    G = graph.toNetworkx()
    distances, paths = nx.single_source_dijkstra(G, source = startNode.name)
    return distances, paths
```

8. ساخت، اضافه کردن و خواندن از LOG فایل:

این توابع مراحل ساخت و اضافه کردن داده به یک فایل متنی و خواندن از همان فایل متنی را نشان می‌دهد. که در نهایت هنگام خواندن مطالب از فایل آن‌ها را با `print()` نیز می‌کنیم تا در terminal نیز نمایش داده شوند. در این توابع برای هر بار اجرای برنامه در یک فایل متنی با نام `LOG.txt` داده‌ها ذخیره می‌شوند و هر بار اجرا این فایل را `rewrite` می‌کند.

```
# create LOG file
def createDataFile(graphInfo):
    # writing data to a text file (start with graph's info)
    with open(outputPath, "w") as file:
        file.write("Graph's info:\n")
        for atmInfo in graphInfo:
            file.write(f"ATM: {atmInfo[0]}, {'Priority ATM' if atmInfo[1] else 'Non Priority ATM'}, Neighbors list: {atmInfo[2]}\n")

# appending data into the LOG file
def addData(refillData, refillStart, results, avgTime, robberyData, robberyStart, rResults, rAvgTime):
    # appending data to the text file
    with open(outputPath, "a") as file:
        # refill data
        file.write("\nRefilling process:\n")
        file.write(f"Start ATM: {refillStart}\n")

        for atmInfo in refillData:
            file.write(f"ATM: {atmInfo[0]}, Refilled at time: {atmInfo[1]:.2f}, Distance: {atmInfo[2]}, Path taken: {atmInfo[3]}\n")

        file.write("\nRefilling results:\n")
        file.write(f"ATMs in order of refilling: {results}\n")
        file.write(f"Average time to refill all ATMs: {avgTime:.2f} hours\n")

        # robbery data
        file.write("\nRobbery process:\n")
        file.write(f"Start ATM: {robberyStart}\n")

        for atmInfo in robberyData:
            file.write(f"ATM: {atmInfo[0]}, Refilled at time: {atmInfo[1]:.2f}, Distance: {atmInfo[2]}, Path taken: {atmInfo[3]}\n")

        file.write("\nRobbery results:\n")
        file.write(f"ATMs in order of robbing: {rResults}\n")
        file.write(f"Average time to rob all ATMs: {rAvgTime:.2f} hours")

# reading and printing the LOG file
def readData():
    # open the file in read mode with UTF-8 encoding
    with open(outputPath, 'r', encoding = 'utf-8') as file:
        for line in file:
            # remove only the trailing newline character and print the line
            print("")
            print(line.rstrip())
```

9. process اصلی پرکردن دستگاه‌های خودپرداز:

برای انجام فرآیند پر کردن دستگاه‌های خودپرداز اول باید یک لیستی از داده‌های آن داشته باشیم که برای ذخیره در LOG فایل است. سپس از نود شروع در زمان $time = 0$ شروع می‌کنیم. نودها به دو بخش `priority` و `nonPriority` تقسیم می‌شوند و به ترتیب اول اولویت دارها و سپس بدون اولویت‌ها پر می‌شوند. روند کاری برای هردو یکسان است. در هر دو دسته تا زمانی که دسته و لیست ما خالی نشود ادامه می‌دهیم به پر کردن دستگاه‌ها خودپرداز. برای این کار ابتدا از نودی که در آن قرار داریم کوتاه‌ترین مسیر را به بقیه نودها توسط تابع `findShortestPath()` شروع می‌کنیم. سپس از میان کوتاه‌ترین مسیرهای به تمامی نودها نودی را که فاصله آن از بقیه کمتر است انتخاب می‌کنیم و از آن شروع می‌کنیم به پر کردن. کامیون پر از پول با سرعت 60 km/h در حال حرکت است، پس بعد از بدست آوردن زمان رسیدن به ATM خالی، آن را با میزان زمان مورد نیاز برای پر کردن دستگاه جمع می‌کنیم و تایم پر شدن دستگاه مشخص می‌شود. سپس تغییرات لازم را در ATM پر شده انجام می‌دهیم. نود فعلی بعدی ما نودی است که تازه پر شده. این کار تا برای هردو دسته از نودها آنقدر ادامه می‌دهیم تا ATM خالی وجود نداشته باشد. از آن جایی که می‌توان این گراف را گراف پراکنده حساب کرد پس با توجه به پیدا کردن نود و تابع `findShortestPath()` مرتبه زمانی هردو حلقه `while` یعنی مرتبه زمانی این بخش از پر کردن دستگاه‌ها برابر است با $O(n(V + E) \log V)$ (پیدا کردن $\min$ در بین کوتاه‌ترین مسیرها n بار اجرا می‌شود).

```
# simulating the delivery and robbery
def process(graph, startATM):
    refillData = [] # refilling data for storing in a LOG file

    current = startATM # refilling current node
    time = 0 # total time for delivery (in hours)
    results = [] # store the delivery results

    priority = [atm for atm in graph.nodes if atm.priority]
    nonPriority = [atm for atm in graph.nodes if not atm.priority]

    while priority:
        # run dijkstra from the current node
        distances, paths = findShortestPath(graph, current)
        next = min(priority, key = lambda atm: distances[atm.name]) # nearest neighbor

        arrivalTime = distances[next.name] / 60 # assuming 60 km/h
        time += arrivalTime + 0.5 # time needed for refilling (30 minutes)

        next.filled = True
        next.refilledTime = time
        current = next

        refillData.append((next.name, time, distances[next.name], paths[next.name])) # add to the refill data
        results.append(next.name) # add the visited node to the list
        priority.remove(next) # remove from the visited list

    while nonPriority:
        # run dijkstra from the current node
        distances, paths = findShortestPath(graph, current)
        next = min(nonPriority, key = lambda atm: distances[atm.name]) # nearest neighbor

        arrivalTime = distances[next.name] / 60 # assuming 60 km/h
        time += arrivalTime + 0.5 # time needed for refilling (30 minutes)

        next.filled = True
        next.refilledTime = time
        current = next

        refillData.append((next.name, time, distances[next.name], paths[next.name])) # add to the refill data
        results.append(next.name) # add the visited node to the list
        nonPriority.remove(next) # remove from the visited list
```

10. process اصلی دزدی از دستگاه‌های خودپرداز:

دزدی از دستگاه‌های پر شده نیز با منطق پر کردن دستگاه‌ها جلو می‌رود و الگوریتم اجرایی برای هردو بهینه است (از لحاظ طی کرد مسیر کمتر). اما با این تفاوت که دزدها پس از 3 ساعت شروع به دزدیدن می‌کنند و هر بار لیستی که ما در حال بررسی آن هستیم تا دستگاه خودپردازی را انتخاب و از آن دزدی کنیم، شامل ATM‌هایی می‌شود که در هر بار اجرای حلقه تایم پر شدن آن‌ها کمتر از تایم کلی دزدی باشد. یعنی قبل از سر رسیدن دزدها پر شده باشد. سپس هر بار چک می‌کنیم که اگر تایم دزدی کردن به طور کلی از تایم اتمام شدن پر کردن دستگاه‌ها بیشتر شود از حلقه خارج و دستگاه خالی نمی‌شود. زیرا با اتمام پر شدن تمامی دستگاه‌ها دزدی تمام می‌شود. سپس میانگین زمانی برای هم پر کردن و هم دزدی کردن محاسبه می‌شود. مرتبه زمانی این بخش نیز مانند حالت‌های پر کردن دستگاه‌ها می‌باشد پس کل تابع `process()` مرتبه زمانی برابر با $O(n(V + E) \log V)$ دارد.

```
robberyData = [] # robbery data for storing in a LOG file

rCurrent = random.choice(graph.nodes) # robbery start node
rTime = 3 # total time for robbing (in hours) (starts at time + 3)
rResults = [] # store robbed ATMs

filled = [atm for atm in graph.nodes if atm.filled and (atm.refilledTime <= rTime)]

while filled:
    filled = [atm for atm in graph.nodes if atm.filled and (atm.refilledTime <= rTime)] # update the filled list

    # run dijkstra from the current node
    distances, paths = findShortestPath(graph, rCurrent)
    rNext = min(filled, key = lambda atm: distances[atm.name]) # nearest neighbor

    arrivalTime = distances[rNext.name] / 100 # assuming 100 km/h
    rTime += arrivalTime + 1 # time needed for robbing an ATM (1 hour)

    # when refilling process ends, robbery should stop
    if (rTime >= time):
        break

    rCurrent = rNext
    rNext.filled = False

    robberyData.append((rNext.name, rTime, distances[rNext.name], paths[rNext.name])) # add to the robbery data
    rResults.append(rNext.name) # add the robbed node to the list
    filled.remove(rNext) #remove from the filled list

# calculate and display average time
avgTime = time / len(results)
# calculate and display average time
if len(rResults) > 0:
    rAvgTime = rTime / len(rResults)
else:
    rAvgTime = 0

addData(refillData, startATM, results, avgTime, robberyData, rCurrent, rResults, rAvgTime)
```

11. بخش اصلی برنامه و ترتیب اجرای هر تابع:

بخش اصلی برنامه ترتیب اجرای تابع‌ها را نشان می‌دهد. ابتدا یک گراف را به صورت تصادفی ایجاد می‌کنیم. بعد چند نود تصادفی را اولویت‌دار می‌کنیم و نود شروع را نیز (که می‌تواند اولویت‌دار یا بدون اولویت باشد) رندوم انتخاب می‌کنیم. سپس مسیر ذخیره LOG فایل را انتخاب می‌کنیم و اطلاعات گراف را در آن ذخیره می‌کنیم. بعد از این اجرای برنامه اصلی شروع می‌شود. با اتمام فرآیند اطلاعات فرآیندهای انجام شده از LOG فایل خوانده شده و نمایش داده می‌شود. سپس گراف به صورت گرافیکی نمایش می‌شود.

```
# creating a graph with 5-15 nodes
graph = generateGraph(random.randint(5, 15))

# assign priority to 2-4 random ATMs
priorityNodes = random.sample(graph.nodes, random.randint(2, 4))
for atm in priorityNodes:
    atm.priority = True

# start the delivery process from a random ATM
startATM = random.choice(graph.nodes)
startATM.startATM = True

# define the path for the LOG file
currentDir = os.path.dirname(os.path.abspath(__file__))
outputPath = os.path.join(currentDir, "LOG.txt")

# storing graph's info
storingGraphsInfo(graph)

# refilling and robbery process
process(graph, startATM)

# printing LOG file
readData()

# visualize the graph
visualizeGraph(graph)
```